

Assessment Brief

Summative Assessment Identification

Programme	BSc (Hons) Software Engineering
Module title	Introduction to Programming and Databases
Academic level / year	1
Teaching period	August to October 2026
Assessment title	Campus MakerSpace Checkout System
Faculty	Chris Bergue

1. Assessment Purpose and Context

This summative Individual Coding Lab evaluates your ability to design and implement a small object-oriented Python application that stores and manages data in a relational database (SQLite). It builds on earlier Python foundations and focuses on modelling real entities with classes, persisting records through SQL operations, validating user input, and presenting a clear menu-driven interface.

The lab matters because professional software commonly combines OOP design with durable data storage. Completing it prepares you for fuller database and application work later in the module and in subsequent software engineering courses. You will also demonstrate your solution live.

2. Learning Outcomes Assessed

Learning outcome	How the assessment provides evidence
LO1: Apply Python programming foundations to build a working software application	Working CLI application with implemented features, validation, and a successful live demo.
LO2: Use object-oriented programming concepts to model a problem domain	Class design (attributes/methods), object collaboration, and clear separation of responsibilities in code.
LO3: Use a database to store, retrieve, update and query application data	SQLite schema, CRUD operations, and at least two meaningful query/report features.
LO4: Communicate technical work through documentation and demonstration	README, inline comments, repository structure, and live feature/code walkthrough.

3. Assessment Task

You have been asked to build a Campus MakerSpace Checkout System for a student makerspace. Members borrow equipment (3D-printer accessories, soldering kits, cameras, laptops, etc.). Your system must register members and equipment, create and close loans, prevent invalid actions, and produce useful status reports from the database.

Build an individual Object-Oriented Python CLI application backed by SQLite. Your programme must allow operators to manage members, equipment inventory, and loans through a clear menu. Data must persist in a database file between runs. You will explain and demonstrate the solution live.

4. Required Deliverables

Submit the following. There is no video deliverable; demonstration is live.

- **Public GitHub repository:** Complete project (source code, database schema/script or auto-created DB on first run, sample data approach, and README). Repository name format: Lab_MakerSpace_{GitHubUsername}
- **Working Python application:** Runs locally and fulfills the required OOP + SQLite features listed in Section 5.
- **Live demonstration:** Scheduled in class / coaching slot. Show all menu features running against the database and walk through your class design and SQL interactions.

5. Task Requirements and Expected Structure

Recommended Project Structure

`main.py` — menu loop and application entry point

`models.py` — domain classes (e.g., Member, Equipment, Loan)

`database.py` — SQLite connection, schema creation, and SQL helpers

`services.py` (optional) — business operations coordinating models and database

`README.md` — how to install/run; overview of classes and tables

`requirements.txt` (if needed) — keep dependencies minimal; `sqlite3` is standard library

Technical Requirements

1. Required OOP Design

- **Classes:** Define at least three meaningful classes with attributes and methods (for example Member, Equipment, Loan).
- **Behaviour:** Encapsulate behaviour in methods (e.g., borrow, return, is_available, overdue helpers).
- **Collaboration:** Show object collaboration (objects used together rather than one long unrelated script).

2. Required Database Features (SQLite)

- **Database file:** Create and use a SQLite database (e.g., `makerspace.db`).
- **Tables:** Provide tables for members, equipment, and loans (normalised enough to avoid duplicating all fields everywhere).
- **CRUD:** Implement Create, Read, Update, Delete (or equivalent status updates) for core records.
- **Reports:** Implement at least two query/report features (e.g., currently borrowed items; overdue loans; equipment by category; member loan history).

3. Required Application Features

- **Register / list / update members**
- **Register / list / update equipment** (including availability status)
- **Create a loan (checkout)** with validation (member exists, equipment available)
- **Return a loan** and update equipment status
- **Search** by member name/ID or equipment name/ID
- **Produce at least two reports** from SQL queries
- **Robust input validation** and clear error messages (no crashes on bad input)

4. Live Demo Requirements

- **Run live:** Demonstrate the full menu workflow end-to-end.
- **Explain design:** Briefly explain your class design and how data is stored/retrieved in SQLite.
- **Q&A:** Be prepared to answer questions about your code and design decisions.

6. Process, Milestones and Checkpoints

Stage	Required evidence or activity	Purpose
Stage 1: Design	Define a simple class sketch, SQLite table list, and menu options.	Confirms OOP and database design before coding.
Stage 2: Implementation	Build classes, schema, CRUD operations, validation, and reports.	Produces a working OOP + SQLite application.
Stage 3: Live Demo & Submission	Push GitHub repo, submit link on Canvas, and complete the live demonstration.	Verifies execution, features, and understanding.

7. Use of Generative AI and Digital Tools

Generative AI tools are permitted with acknowledgement as learning and debugging aids.

- **Allowed:** Explain errors, discuss design options, or improve documentation.
- **Not allowed:** Submitting an AI-generated whole application that you cannot explain in the live demo.
- **Disclosure:** If AI helped write or fix non-trivial code, disclose it in your README (tool used and what it helped with).

8. Academic Integrity and Referencing

This is an individual assessment. Collaboration on ideas is allowed; sharing or copying code is not.

- **Authorship:** All code must be your own work for this summative.
- **Attribution:** Attribute any adapted snippets with comments and links.
- **Demo accountability:** Submitting work you cannot explain during the live demo will be handled under institutional academic integrity procedures.
- **Referencing:** APA 7th is recommended for external sources listed in your README.

9. Marking Criteria and Rubric

Total: 100 points. Criteria align with the required OOP design, SQLite database use, application features, validation, live demonstration, and submission requirements.

Criterion	Weight	Exemplary	Proficient	Developing	Beginning	No Marks
Object-Oriented Programming Design	20	20-17 pts: At least three well-designed classes with clear attributes/methods, sensible encapsulation, and effective object collaboration.	16-13 pts: Required classes are present and mostly well structured; collaboration is evident with only minor design weaknesses.	12-8 pts: Classes exist but design is shallow (limited methods, weak encapsulation, or mostly procedural code wrapped in classes).	7-1 pts: Minimal or incorrect OOP use; little evidence of meaningful class design.	0 pts: No meaningful OOP design.
Database Design and Use (SQLite)	25	25-21 pts: Clear schema for members/equipment/loans, correct CRUD operations, and at least two meaningful SQL reports/queries demonstrated.	20-16 pts: SQLite is used effectively with working CRUD; schema or reports have only minor gaps.	15-10 pts: Database present but incomplete (weak schema, partial CRUD, or missing/poor report queries).	9-1 pts: Minimal database use; data mostly hard-coded or not reliably persisted.	0 pts: No usable SQLite database implementation.
Application Features and Execution	20	20-17 pts: Required menu features run reliably end-to-end (members, equipment, loans, search, reports) with correct results.	16-13 pts: Most required features work correctly; only minor functional or usability issues.	12-8 pts: Several required features incomplete, unstable, or produce incorrect results.	7-1 pts: Few features work; application is largely incomplete or unreliable.	0 pts: Application does not run / core features not implemented.
Error Handling and Validation	15	15-13 pts: Robust validation and clear feedback for invalid input and edge cases (e.g., unavailable equipment, missing member) without crashes.	12-10 pts: Good handling of common errors; a few edge cases remain weakly handled.	9-6 pts: Limited validation; some errors go unhandled or messages are unclear.	5-1 pts: Minimal validation; application is difficult or fragile to use.	0 pts: No usable validation or error handling.
Live Demonstration	15	15-13 pts: Confident live demo of the full workflow; clear explanation of class design and SQLite interactions; strong responses to questions.	12-10 pts: Successful demo of main features with generally clear explanation; minor gaps in Q&A or design discussion.	9-6 pts: Partial demo; limited explanation of OOP/database design or difficulty answering questions.	5-1 pts: Weak demo; cannot reliably run features or explain own code/design.	0 pts: No live demonstration / unable to demonstrate the work.
Following Submission Instructions	5	5 pts: All submission requirements met (repo naming, required files/README, Canvas link, and demo attendance).	4 pts: Minor naming/formatting issues; submission otherwise complete.	3-2 pts: Some required elements missing (e.g., incomplete repo package or partial Canvas submission).	1 pt: Major submission problems (wrong location, missing repository link, or incomplete package).	0 pts: No valid submission.
Total	100					

10. Feedback and Use of Feedback

You will receive rubric-based feedback on Canvas after marking, informed by both your repository and live demo. Use the feedback to improve class design, SQL usage, validation, and demo communication in later assessments.

11. Student Submission Checklist

- ☐ I have completed every required deliverable (GitHub repo + working app).
- ☐ My application uses OOP classes and SQLite as required.
- ☐ I can run and explain all required features in a live demo.
- ☐ I have checked that all files open correctly and use the required repository naming format.
- ☐ I have acknowledged and referenced sources, code snippets and AI assistance where used.
- ☐ I have followed the stated rules for individual authorship and generative AI.
- ☐ I have reviewed the rubric and checked my work against every criterion.

12. Additional Resources

Resource	Purpose	Link / Location
SQLite Python docs	Using sqlite3 from the standard library	Link
Module learning materials	Lectures and labs on OOP and databases	Link
Generative AI guidance	Responsible AI use and disclosure	Link

13. Questions and Clarification

Post clarification questions in the Canvas Discussion Board for this module so all students can see the answers.

For private queries about your code or demo scheduling, contact jbergue@alueducation.com or raise them during coaching sessions.